Introduction to Transact - SQL

Stored Procedures – Functions – Triggers

Baseado nos apontamentos de Sistemas de Bases de Dados - LEI 2013/2014

Introduction

- SQL Server allows you to create three types of programmable objects:
  - Stored Procedures
  - Functions
  - Triggers
- Instead of executing a single statement or command at a time, you can build objects that contain blocks of code using a rich programming language with:
  - Variables
  - Parameters
  - Control flow
  - Error handling
- In addition to the three types of programmable objects, you can store SELECT statements within a database by using views

T-SQL – Basic Concepts

- Commenting Code
  - You should include enough comments within your code to allow subsequent developers to understand what the code does.

```
-- This is a single line comment

/*
This is
a multi-line comment
*/
```

Variables

- SQL Server has two types of variables: local and global;
- A local variable is designated by a single at sign (@), while global variable is designated by double at sign (@@);
- You can create, read and write local variables but you can only read global variables;

Variables

- Global variables:

This variable have limitations. It is recommended to use the SCOPE_IDENTITY() function

Global variable	Definition
@@ERROR	Error from the last statement executed
@@IDENTITY	Value of the last identity value inserted within the connection
@@ROWCOUNT	The number of rows affected by the last statement
@@TRANCOUNT	The number of open transactions within the connection
@@VERSION	The version of SQL Server

Variables: Declaration

```
DECLARE @intvariable INT,
        @datevariable DATE,
        @floatvariable FLOAT,
        @decimalvariable DECIMAL(5, 2),
        @textvariable VARCHAR(50);
```

Declare clause to instantiate a variable

Name of the variable

Data type of the variable
Data Types available:
http://technet.microsoft.com/en-us/library/ms187752.aspx

Variables: Assign values

- At the declaration:

```
DECLARE @intvariable INT = 1,
        @datevariable DATE = GETDATE(),
        @maxorderdate DATE = (SELECT MAX(OrderDate) FROM Orders.OrderHeader),
        @name VARCHAR(50) = 'Database‘;
```

If you are executing a query to assign a value, you must use a SELECT statement.
You can only assign scalar values in variables

```
SET @intvariable = 2;
SELECT @intvariable = 3;
```

Either a SET or a SELECT can be used to assign a value

```
SELECT @intvariable, @datevariable, @maxorderdate, @name
```

Control Flow Constructs

- IF…ELSE
  - Provides the ability to conditionally execute code. The IF statement checks the conditions supplied and executes the next statement when condition evaluates to True. The optional ELSE statement allows you to execute code when the condition check evaluates to False;

```
DECLARE @var SMALLINT;

SET @var = 1;

IF @var = 1
BEGIN
    PRINT 'This is the code executed when true.';
END
ELSE
BEGIN
    PRINT 'This is the code executed when false.';
END
```

The BEGIN..END statement allows you to delimit code blocks that execute as a unit

Control Flow Constructs

- WHILE
  - It is used to iteratively execute a block of code so long as a specified condition is true;

```
DECLARE @var1 SMALLINT,
        @var2 VARCHAR(30);

SET @var1 = 1;

WHILE (@var1 <= 10)
BEGIN
    SET @var2 = 'Iteration #' + CAST(@var1 AS VARCHAR(2));

    PRINT @var2;

    SET @var1 += 1;
END
```

Error Handling

- T-SQL provides a structured way of handling errors that is very similar to the error handling routines of other programming languages: a TRY… CATCH block;

```
BEGIN TRY
    BEGIN TRAN
        INSERT INTO dbo.mytable VALUES(1);
        INSERT INTO dbo.mytable VALUES(1);
        INSERT INTO dbo.mytable VALUES(2);
    COMMIT TRAN
END TRY

BEGIN CATCH
    ROLLBACK TRAN;
    PRINT 'Catch';
    -- You can optionally rethrow the error with "THROW;"
END CATCH
```

Transaction block

Cursors (use with caution)

- With traditional SQL you handle sets of data;
- However, there are many times when you need to process data one row at a time;

```
DECLARE @ProductID INT,
        @ProductName VARCHAR(50),
        @ListPrice DECIMAL(10, 4);
```

DECLARE is used to define the SELECT statement that is the basis for the rows in the cursor

```
DECLARE curproducts CURSOR FOR
    SELECT ProductID, ProductName, ListPrice FROM Products.Product
FOR READ ONLY
```

CURSOR type

```
OPEN curproducts
```

OPEN causes the SELECT statement to be executed and load the rows into a memory structure

```
FETCH curproducts INTO @ProductID, @ProductName, @ListPrice
WHILE (@@FETCH_STATUS = 0)
BEGIN
    SELECT @ProductID, @ProductName, @ListPrice;
    FETCH curproducts INTO @ProductID, @ProductName, @ListPrice;
END
```

FETCH is used to retrieve one row at the time from the cursor

CLOSE is used to close the processing on the cursor

```
CLOSE curproducts
DEALLOCATE curproducts
```

DEALLOCATE is used to remove the cursor and release the memory structures containing the cursor result set

Stored Procedures

- Stored Procedures allow you to make changes to the database structure and manage performance without needing to rewrite applications or deploy application updates;

- The main purpose of a stored procedure is to provide a security layer and a API to your databases that isolate applications from changes to the database structure;

- Executing stored procedures:

```
EXEC <stored procedure> <param1>, … ,<paramn>
```

Stored procudure name

Parameters (by position)

Stored procedures - Parameters

- Parameters are local variables that are used to pass values into a stored procedure when it is executed;
- During the execution any parameters can be read and written;
- Example:

```
CREATE PROCEDURE usp_GetEmployeeByName
        @Lastname VARCHAR (50),
        @Firstname VARCHAR (50)
AS
SET NOCOUNT ON
```

This statement turns off messages that SQL Server sends back to the client after any SELECT, INSERT, UPDATE, MERGE, and DELETE statements are executed. Overall performance of the database and application is improved by eliminating this unnecessary network overhead

- http://technet.microsoft.com/en-us/library/ms187926.aspx

Stored Procedure - Example

```
--Stored procedure example
CREATE PROCEDURE usp_UpdateJobRoleHireDateEmployee
	-- Add the parameters for the stored procedure here
	 @jobRole VARCHAR (50)
AS
BEGIN

	SET NOCOUNT ON

	DECLARE @now DATETIME = GETDATE()
	 -- Insert statements for procedure here
	BEGIN TRAN
	BEGIN TRY
		UPDATE Employee
			SET Employee.HireDate = @now
		WHERE Employee.JobRole = @jobRole
		COMMIT TRAN
	END TRY
	BEGIN CATCH
		ROLLBACK TRAN
		PRINT 'An error occured, transaction rolled back'
	END CATCH
END
```

User-Defined Functions

- Functions are programmable objects that are used to perform calculations that can be returned to a calling application or integrated into a result set;
- Functions can access the data and return results, but they cannot make any modifications;
- Every function ends with RETURN statement;
- You retrieve data from a function by using a SELECT statement;
- Functions in CHECK and DEFAULT constraints are used to extend the static computations available;
- For example, you can validate the area code for a phone number against a list of area codes stored within a table;
- Some system functions: http://technet.microsoft.com/en-us/library/ms174318.aspx

Scalar Function: Example

```
CREATE FUNCTION ufi_GetLocality (@Code INT)
RETURNS VARCHAR(50)
AS
BEGIN

	DECLARE @Locality VARCHAR(50)

	SELECT @Locality = Location.Locality
		FROM Location
		WHERE Location.Code = @Code

	IF(@Locality IS NULL)
	BEGIN
		SET @Locality = 'Unknown locality'
	END

	RETURN @Locality

END
```

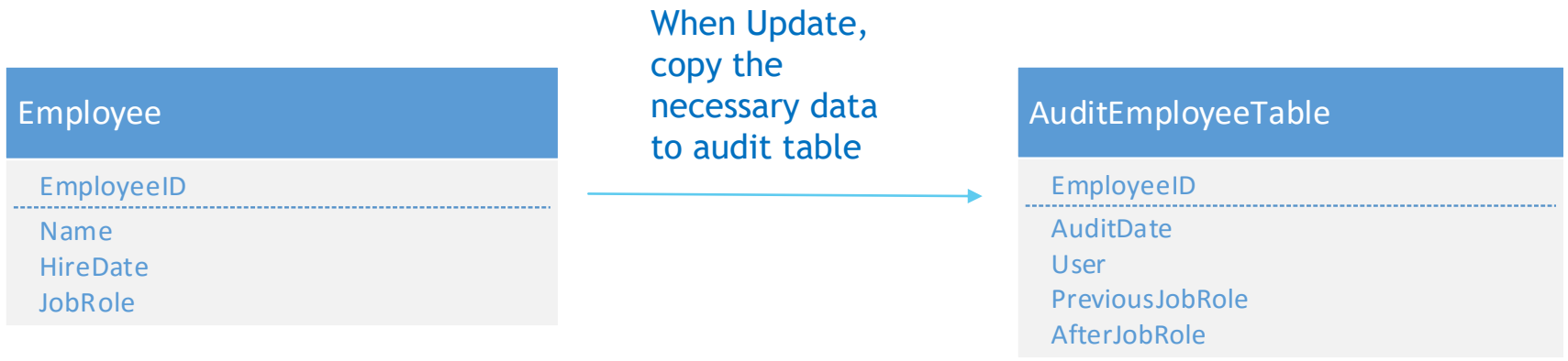
Return results from the function: `SELECT dbo.GetLocality('4610')`

Triggers

- Triggers are a special type of stored procedures that automatically execute when a DML or DDL statement associated with the triggers is executed;
- The DML triggers execute when you add, modify, or remove rows in a table;
- The DDL triggers execute when DDL commands are executed or users log in to a SQL server instance;
- DML triggers are created against a table or a view and are defined for a specific event: INSERT, UPDATE, or DELETE;
- AFTER triggers fires after the modification has passed all constraints;
- A trigger defined with INSTEAD OF clause causes the trigger code to be executed as a replacement for INSERT, UPDATE, or DELETE

Triggers: Example

- DML Trigger – Maintain an audit table
- What is the purpose?
- For example, every time an employee job role is updated, we want to trail the changes in a different table, in order to perverse all job roles from the employees:



Triggers: Example

```
CREATE TRIGGER [EmployeeUpdateAudit]
  ON  [Employee]
  AFTER UPDATE
AS
BEGIN
        SET NOCOUNT ON;

        DECLARE @EmployeeID INT,
                @PreviousJobRole VARCHAR(50),
                @AfterJobRole VARCHAR(50)

        SELECT @EmployeeID = i.EmployeeID, @PreviousJobRole= d.JobRole, @AfterJobRole = i.JobRole
        FROM inserted i JOIN deleted d ON i.EmployeeID = d.EmployeeID

        IF(@@ROWCOUNT > 0) AND (@PreviousJobRole != @AfterJobRole)
        BEGIN
                INSERT INTO AuditEmployeeTable
                (EmployeeID
                ,AuditDate
                ,User
                ,PreviousJobRole
                 ,AfterJobRole)
                 VALUES
                (@EmployeeID
                 ,GETDATE()
                ,SUSER_SNAME()
                ,@PreviousJobRole
                ,@AfterJobRole
                )
        END
END
```

Assign values to local variables

- The deleted table stores copies of the affected rows during DELETE and UPDATE statements
- The inserted table stores copies of the affected rows during INSERT and UPDATE statements.
- An update transaction is similar to a delete operation followed by an insert operation;

Triggers: Example

- Test the trigger.
- State of EmployeeTable:

EmployeeID	Name	HireDate	JobRole
4	Bruno	2013-12-16 15:04:56.060	C
5	Ana	2013-12-16 14:46:17.860	B
6	João	2013-12-16 15:02:36.363	C

- Update Data:

```
UPDATE Employee
      SET JobRole = 'A'
WHERE EmployeeID = 5
```

- AuditEmployeeTable state, after the update:

EmployeeID	AuditDate	User	PreviousJobRole	AfterJobRole
5	2013-12-16 16:57:00.790	LG\Bruno	B	A

Triggers: Example

- Now, we are going to update all employees from jobRole C to jobRole D

EmployeeID	Name	HireDate	JobRole
4	Bruno	2013-12-16 15:04:56.060	D
5	Ana	2013-12-16 14:46:17.860	A
6	João	2013-12-16 15:02:36.363	D

Two rows were updated

- How AuditEmployeeTable was updated?

EmployeeID	AuditDate	User	PreviousJobRole	AfterJobRole
4	2013-12-16 16:57:00.790	LG\Bruno	C	A
4	2013-12-16 21:30:50.657	LG\Bruno	C	D

The new one

- AuditEmployeeTable is correctly updated?

Triggers: Example

- Regardless of the number of rows that are affected, a trigger fires only once for an action;

- We need a statement level trigger, instead of a row-level trigger that we already have:

```
CREATE TRIGGER EmployeeUpdateAudit
   ON  Employee
   AFTER UPDATE
AS
BEGIN
      SET NOCOUNT ON;
      IF @PreviousJobRole != @AfterJobRole
      BEGIN
            INSERT INTO AuditEmployeeTable
            SELECT i.EmployeeID, GETDATE(), SUSER_SNAME(), d.JobRole, i.JobRole
            FROM inserted i JOIN deleted d ON i.EmployeeID = d.EmployeeID
      END
END
```

References

- http://technet.microsoft.com/pt-pt/sqlserver/ff898410

- Tobias Thernström, Ann Weber, Mike Hotek, and GrandMasters. MCTS Self-Paced Training Kit (Exam 70-433): Microsoft® SQL Server® 2008 - Database Development. 2009